A2P3

March 4, 2025

1 Problem 3 part (a)

[52]:
```
from gurobipy import *
model = Model("A2P3")
model.setParam('OutputFlag', 0)

##SETS
# P = Set of all Products
P = ["Prod1", "Prod2", "Prod3"]

##PARAMETERS
# c_p = profit per unit for each product p in set P
# t_p = time required per unit to produce each product p in set P
# d_p = maximum demand of product p in set P
# b_p = minimum production required of product p in set P
# L = Limit on total time required to product ALL products p in the set P
c_p = {
    "Prod1": 20,
    "Prod2": 40,
    "Prod3": 32
}

t_p = {
    "Prod1": 4.6,
    "Prod2": 7.5,
    "Prod3": 2.4
}

d_p = {
    "Prod1": 102.3,
    "Prod2": 357.4,
    "Prod3": 182.6
}

b_p = {
    "Prod1": 15.3,
    "Prod2": 23.2,
```

```
    "Prod3": 17.5
}
L = 556.7


##DECISION VARIABLES
# x_p = total number of units produced of product p in set P
x_p={}
for p in P:
    x_p[p]=model.addVar(vtype=GRB.CONTINUOUS, name="x_" + p)

##OBJECTIVE FUNCTION
# MAXIMIZE the total profit of the production system
z = quicksum(c_p[p] * x_p[p] for p in P)
model.setObjective(z,GRB.MAXIMIZE)


##CONSTRAINTS
# (1) Maximum production (maximum demand) constraint
# (2) Minimum production constraint
# (3) Total production time constraint
# (4) Non-negativity constraint
    # Already accounted for in definition of decision variables
model.addConstrs(x_p[p] <= d_p[p] for p in P) #1
model.addConstrs(x_p[p] >= b_p[p] for p in P) #2
model.addConstr(quicksum(t_p[p]*x_p[p] for p in P) <= L) #3


##SOLVE AND PRINT
model.update()
model.optimize()

if model.status == GRB.OPTIMAL:
    print("Optimal solution found!")
    print("Optimal objective value (Total profit), z* =", model.objVal)
    for p in P:
        print(f"x*_{p } = {x_p[p].x}")
else:
    print("No optimal solution found.")
```

```
Optimal solution found!
Optimal objective value (Total profit), z* = 5398.266666666667
x*_Prod1 = 15.3
x*_Prod2 = 23.2
x*_Prod3 = 130.13333333333335
```

1.0.1 Solution is non-integer, and requires further manipulation via branch-and-bound to obtain the optimal solution.

Branching on Prod1

Branch 1: x_Prod1 <= 15

[53]:
```
## PULL PREVIOUS MODEL
branch_left = model.copy()
branch_left.update()

## DEFINE DECISION VARIABLE
x_prod1 = branch_left.getVarByName("x_Prod1")

## ADD CONSTRAINT
branch_left.addConstr(x_prod1 <= 15)

branch_left.optimize()

if branch_left.status == GRB.OPTIMAL:
    print("Branch: x_Prod1 <= 15")
    for p in P:
        var = branch_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")
```

```
No optimal solution found in this branch.
```

Branch 2: x_Prod2 >= 16

[54]:
```
## PULL PREVIOUS MODEL
branch_right = model.copy()
branch_right.update()

## DEFINE DECISION VARIABLE
x_prod1 = branch_right.getVarByName("x_Prod1")

## ADD CONSTRAINT
branch_right.addConstr(x_prod1 >= 16)

branch_right.optimize()

if branch_left.status == GRB.OPTIMAL:
    print("Branch: x_Prod1 >= 16")
    for p in P:
        var = branch_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
```

```
else:
    print("No optimal solution found in this branch.")
```

```
No optimal solution found in this branch.
```

Branching on Prod2

Branch 1: x_Prod2 <= 23

[55]:
```
## PULL PREVIOUS MODEL
branch_left = model.copy()
branch_left.update()

## DEFINE DECISION VARIABLE
x_prod2 = branch_left.getVarByName("x_Prod2")

## ADD CONSTRAINT
branch_left.addConstr(x_prod2 <= 23)

branch_left.optimize()

if branch_left.status == GRB.OPTIMAL:
    print("Branch: x_Prod2 <= 23")
    for p in P:
        var = branch_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")
```

```
No optimal solution found in this branch.
```

Branch 2: x_Prod2 >= 24

[56]:
```
## PULL PREVIOUS MODEL
branch_right = model.copy()
branch_right.update()

## DEFINE DECISION VARIABLE
x_prod2 = branch_right.getVarByName("x_Prod2")

## ADD CONSTRAINT
branch_right.addConstr(x_prod2 >= 24)

branch_right.optimize()

if branch_left.status == GRB.OPTIMAL:
    print("Branch: x_Prod2 >= 24")
    for p in P:
```

```
        var = branch_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")
```

```
No optimal solution found in this branch.
```

Branching on Prod3

Branch 1: x_Prod3 <= 130

[57]:
```
## PULL PREVIOUS MODEL
branch_left = model.copy()
branch_left.update()

## DEFINE DECISION VARIABLE
x_prod3 = branch_left.getVarByName("x_Prod3")

## ADD CONSTRAINT
branch_left.addConstr(x_prod3 <= 130)

branch_left.optimize()

if branch_left.status == GRB.OPTIMAL:
    print("Branch: x_Prod3 <= 130")
    for p in P:
        var = branch_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")
```

```
Branch: x_Prod3 <= 130
Prod1: 15.3
Prod2: 23.242666666666672
Prod3: 130.0
```

Branch 2: x_Prod3 >= 131

[58]:
```
## PULL PREVIOUS MODEL
branch_right = model.copy()
branch_right.update()

## DEFINE DECISION VARIABLE
x_prod3 = branch_right.getVarByName("x_Prod2")

## ADD CONSTRAINT
branch_right.addConstr(x_prod3 >= 131)
```

```
branch_right.optimize()

if branch_left.status == GRB.OPTIMAL:
    print("Branch: x_Prod3 >= 131")
    for p in P:
        var = branch_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")
```

```
Branch: x_Prod3 >= 131
Prod1: 15.3
Prod2: 23.242666666666672
Prod3: 130.0
```

The results indicate that the products Prod1 and Prod2 can not be initially branched upon to approach the optimal integral solution. Prod3 assumes an integer value of 130.00 on both bounds of the associated branch. From here, we can create a new sub-model to further explore integrality on the remaining products. Remaining exploration of branches will be condensed into single cells for each variable.

Prod3 integral—Branching on Prod1

[59]:
```
## PULL PREVIOUS MODEL
itertwo_branching = branch_left.copy()
itertwo_branching.update()

## DEFINE DECISION VARIABLE
x_prod1 = itertwo_branching.getVarByName("x_Prod1")

## LEFT BRANCH
itertwo_branching.addConstr(x_prod1 <= 15)
itertwo_branching.optimize()

if itertwo_branching.status == GRB.OPTIMAL:
    print("Branch: x_Prod1 <= 15")
    for p in P:
        var = itertwo_branching.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")

## RIGHT BRANCH
itertwo_branching.addConstr(x_prod1 >= 16)
itertwo_branching.optimize()

if itertwo_branching.status == GRB.OPTIMAL:
    print("Branch: x_Prod1 <= 16")
```

```
    for p in P:
        var = itertwo_branching.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")
```

```
No optimal solution found in this branch.
No optimal solution found in this branch.
```

Prod3 integral—Branching on Prod2

[60]:
```
## PULL PREVIOUS MODEL
itertwo_branching = branch_left.copy()
itertwo_branching.update()

## DEFINE DECISION VARIABLE
x_prod2 = itertwo_branching.getVarByName("x_Prod2")

## LEFT BRANCH
itertwo_branching.addConstr(x_prod2 <= 23)
itertwo_branching.optimize()

if itertwo_branching.status == GRB.OPTIMAL:
    print("Branch: x_Prod2 <= 23")
    for p in P:
        var = itertwo_branching.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")

## RIGHT BRANCH
itertwo_branching.addConstr(x_prod2 >= 24)
itertwo_branching.optimize()

if itertwo_branching.status == GRB.OPTIMAL:
    print("Branch: x_Prod2 <= 24")
    for p in P:
        var = itertwo_branching.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in this branch.")
```

```
No optimal solution found in this branch.
No optimal solution found in this branch.
```

Branching on Prod1 and Prod2 fails, because the floor value of each solution to the relaxed LP fails to satisfy the minimum production constraint. How is the optimal solution to the LP affected if b_p and d_p are pre-processed to assume integer values? (Ceiling of b_p, Floor of d_p)

b_p = { “Prod1”: 15.3 —> 16, “Prod2”: 23.2 —> 24, “Prod3”: 17.5 —> 18 }

d_p = { “Prod1”: 102.3 —> 102, “Prod2”: 357.4 —> 357, “Prod3”: 182.6 —> 182 }

[61]:
```
from gurobipy import *
model = Model("A2P3Preprocessed")
model.setParam('OutputFlag', 0)

##SETS
# P = Set of all Products
P = ["Prod1", "Prod2", "Prod3"]

##PARAMETERS
# c_p = profit per unit for each product p in set P
# t_p = time required per unit to produce each product p in set P
# d_p = maximum demand of product p in set P
# b_p = minimum production required of product p in set P
# L = Limit on total time required to product ALL products p in the set P
c_p = {
    "Prod1": 20,
    "Prod2": 40,
    "Prod3": 32
}

t_p = {
    "Prod1": 4.6,
    "Prod2": 7.5,
    "Prod3": 2.4
}

d_p = {
    "Prod1": 102,
    "Prod2": 357,
    "Prod3": 182
}

b_p = {
    "Prod1": 16,
    "Prod2": 24,
    "Prod3": 18
}
L = 556.7


##DECISION VARIABLES
# x_p = total number of units produced of product p in set P
x_p={}
for p in P:
```

```
    x_p[p]=model.addVar(vtype=GRB.CONTINUOUS, name="x_" + p)

##OBJECTIVE FUNCTION
# MAXIMIZE the total profit of the production system
z = quicksum(c_p[p] * x_p[p] for p in P)
model.setObjective(z,GRB.MAXIMIZE)


##CONSTRAINTS
# (1) Maximum production (maximum demand) constraint
# (2) Minimum production constraint
# (3) Total production time constraint
# (4) Non-negativity constraint
    # Already accounted for in definition of decision variables
model.addConstrs(x_p[p] <= d_p[p] for p in P) #1
model.addConstrs(x_p[p] >= b_p[p] for p in P) #2
model.addConstr(quicksum(t_p[p]*x_p[p] for p in P) <= L) #3


##SOLVE AND PRINT
model.update()
model.optimize()

if model.status == GRB.OPTIMAL:
    print("Optimal solution found!")
    print("Optimal objective value (Total profit), z* =", model.objVal)
    for p in P:
        print(f"x*_{p } = {x_p[p].x}")
else:
    print("No optimal solution found.")
```

```
Optimal solution found!
Optimal objective value (Total profit), z* = 5321.333333333334
x*_Prod1 = 16.0
x*_Prod2 = 24.0
x*_Prod3 = 126.29166666666669
```

[75]:
```
## PULL PREVIOUS MODEL for left branch
iterthree_branching_left = model.copy()
iterthree_branching_left.update()
x_prod3_left = iterthree_branching_left.getVarByName("x_Prod3")

## LEFT BRANCH: force x_Prod3 <= 126
iterthree_branching_left.addConstr(x_prod3_left <= 126,␣
 ↪name="branch_left_x_Prod3")
iterthree_branching_left.optimize()
```

```
if iterthree_branching_left.status == GRB.OPTIMAL:
    print("Left Branch: x_Prod3 <= 126")
    for p in P:
        var = iterthree_branching_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in the left branch.")

## PULL PREVIOUS MODEL for right branch
iterthree_branching_right = model.copy()
iterthree_branching_right.update()
x_prod3_right = iterthree_branching_right.getVarByName("x_Prod3")

## RIGHT BRANCH: force x_Prod3 >= 127
iterthree_branching_right.addConstr(x_prod3_right >= 127,␣
 ↪name="branch_right_x_Prod3")
iterthree_branching_right.optimize()

if iterthree_branching_right.status == GRB.OPTIMAL:
    print("Right Branch: x_Prod3 >= 127")
    for p in P:
        var = iterthree_branching_right.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in the right branch.")
```

```
Left Branch: x_Prod3 <= 126
Prod1: 16.0
Prod2: 24.093333333333344
Prod3: 126.0
No optimal solution found in the right branch.
```

[76]:
```
iterfour_branching_left = iterthree_branching.copy()
iterfour_branching_left.update()
x_prod2_left = iterfour_branching_left.getVarByName("x_Prod2")

## LEFT BRANCH: force x_Prod2 <= 24
iterfour_branching_left.addConstr(x_prod2_left <= 24, name="branch_left_x_Prod2")
iterfour_branching_left.optimize()

if iterfour_branching_left.status == GRB.OPTIMAL:
    print("Left Branch: x_Prod2 <= 24")
    for p in P:
        var = iterfour_branching_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in the left branch.")
```

```
iterfour_branching_right = iterthree_branching.copy()
iterfour_branching_right.update()
x_prod2_right = iterfour_branching_right.getVarByName("x_Prod2")

## RIGHT BRANCH: force x_Prod2 >= 25
iterfour_branching_right.addConstr(x_prod2_right >= 25,␣
↪name="branch_right_x_Prod2")
iterfour_branching_right.optimize()

if iterfour_branching_right.status == GRB.OPTIMAL:
    print("Right Branch: x_Prod2 >= 25")
    for p in P:
        var = iterfour_branching_right.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in the right branch.")
```

```
Left Branch: x_Prod2 <= 24
Prod1: 16.152173913043494
Prod2: 24.0
Prod3: 126.0
Right Branch: x_Prod2 >= 25
Prod1: 16.0
Prod2: 25.0
Prod3: 123.16666666666669
```

[77]:
```
iterfive_branching_left = iterfour_branching.copy()
iterfive_branching_left.update()
x_prod1_left = iterfive_branching_left.getVarByName("x_Prod1")
# LEFT BRANCH: force x_Prod1 <= 16
iterfive_branching_left.addConstr(x_prod1_left <= 16, name="branch_left_x_Prod1")
iterfive_branching_left.optimize()

if iterfive_branching_left.status == GRB.OPTIMAL:
    print("Left Branch: x_Prod1 <= 16")
    for p in P:
        var = iterfive_branching_left.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in the left branch.")


iterfive_branching_right = iterfour_branching.copy()
iterfive_branching_right.update()
x_prod1_right = iterfive_branching_right.getVarByName("x_Prod1")
```

```
# RIGHT BRANCH: force x_Prod1 >= 17
iterfive_branching_right.addConstr(x_prod1_right >= 17,␣
 ↪name="branch_right_x_Prod1")
iterfive_branching_right.optimize()

if iterfive_branching_right.status == GRB.OPTIMAL:
    print("Right Branch: x_Prod1 >= 17")
    for p in P:
        var = iterfive_branching_right.getVarByName("x_" + p)
        print(f"{p}: {var.X}")
else:
    print("No optimal solution found in the right branch.")
```

```
Left Branch: x_Prod1 <= 16
Prod1: 16.0
Prod2: 24.0
Prod3: 126.0
Right Branch: x_Prod1 >= 17
Prod1: 17.0
Prod2: 24.0
Prod3: 124.37500000000003
```

SOLUTION WHERE Prod1 = 16, Prod2 = 24, Prod3 = 126

[ ]:

(c) Search Strategies

- **Depth-First Search:** First explores one branch until a complete feasible integer solution is found, then backtracks.
- **Best-Node-First Search:** First selects the node with highest LP relaxation bound to explore next.
- **Least-Discrepancy Search:** Follows heuristic ordering suggested by the LP solution, by first exploring nodes that deviate minimally from the LP solution.

(d) Cutting Planes

Cutting planes are inequalities added to the LP relaxation that remove fractional solutions without excluding the feasible integer solutions. Cutting planes tighten the relaxed LP, which improves the bound and can reduce the number of nodes in the branch and bound tree.

(e) Branch and Bound with Cuts

By adding a valid cut derived from the LP solution (for instance, from the fractional part of $Prod3$), the LP relaxation can be tightened. In our case, the cut is designed to eliminate the fractional solution $Prod3 \approx 130.1333$ while preserving all integer feasible points. Re-solving the LP with the cut may yield the integer solution directly:

$$Prod1 = 16, \quad Prod2 = 24, \quad Prod3 = 126, \quad (Objective)\ z = 5312.$$

The branch and bound tree is reduced when cuts are used in combination with branching, highlighting the efficiency of a branch-and-cut approach.